

COL331 Project Report

Mani Sarthak (2021CS10095), Disha Shiraskar (2021CS10578), Naman Nirwan (2021CS50593)

April 24, 2024

1 Introduction

The goal of lab4 and the project is to enhance xv6 by implementing demand paging and copy-on-write (COW) mechanisms. Demand paging enables accessing more memory than physically available, while COW optimizes memory usage by sharing read-only pages among processes. Swapping facilitates accessing data from disk when a requested page is not in physical memory, and COW efficiently copies pages between address spaces without moving data unless necessary. This enhances system performance and memory management in xv6.

2 Implementing Demand Paging

Swapping is implemented using disk blocks as swap space. A swap slot structure is defined to manage swap space, containing page permissions, availability status, and the starting block ID. Swap space is represented as an array of swap slots. Functions for adding/removing pages to/from swap space are defined. The superblock is updated to include swap space information, and swap space initialization is performed in `main.c`.

Other functions were modified as follows:

- `kalloc()` : Previously, `kalloc()` returned 0 if it couldn't allocate memory for a new page. Now, if `kalloc()` fails, it invokes `swap_page()`. This function selects a victim page from a victim process, updates the victim process's page table entry (pte) to store the swap slot address, updates the RSS, marks the swap slot as used, and swaps the page to disk, freeing up space in memory. After this, `kalloc()` retries to find a free page and returns it.
- `trap()` : If a process attempts to access a swapped-out page, then x86 will generate the `*T_PGFLT*` trap signal. We invoke a `swap_page_in()` on receiving this signal. This function extracts swap slot addr from the pte of the page that gave pagefault, allocates a new page using `kalloc()`, transfers the contents of the corresponding swap slot to the new page, extracts permissions from `pageperm` attribute, updates pte of the process to store addr of new page along with the extracted page permissions and sets swap slot as free.
- The rss values of the process were updated wherever required. If process enters a ZOMBIE state all the swap slots corresponding to that process are cleared.

3 Implementing COW

COW meant that more than 1 processes could have page table entry mapped to a particular physical address. To keep a track of all this we defined the structs below:

- `uint rmap[PHYSTOP>>PTXSHIFT]` : This is an array of integers of size equal to number of physical addresses. The entry at index `i` stores the count of the number of processes which have a page table entry corresponding to physical address `i`.
- `pte_t* proc_ref[PHYSTOP>>PTXSHIFT][NPROC]` : This is 2D array of pointers to pte of size (number of physical addresses)*(maximum no. of processes possible). Each element `proc_ref[i][j]`

of the array stores a pointer to a page table entry (`pte_t*`) in the page tables of the j th process that maps to the physical memory address i . If multiple processes have page table entries corresponding to the same physical memory address i , the first j entries of `proc_ref[i]` will be filled with pointers to the respective page table entries of those processes.

- `struct swap_slot` : The existing struct from lab4 was modified to accommodate the scenario where a single physical page is referenced by multiple processes. When a physical page is chosen to be swapped out, the page table entries of all referring processes must be updated. Similarly, if the page is swapped in later, these page table entries should be updated again. To track these page table entries, two additional attributes were added to the struct: `pte_t* processReference[NPROC]` to track all page table entries corresponding to the swapped page, and `reference count` to store the count of processes that have page table entries for the swapped page.

Other functions allowing changes to `rmap` and `pte_t* proc_ref` (addition or removal), saving process reference (when swapping out) and restoring process reference (swapping in) were defined accordingly. Now, it was not enough to update `rss` of single process whenever swapping took place, therefore a separate function has been defined to update `rss` of all the processes referring to the page being swapped in or swapped out. Moreover, `rmap` and `proc_ref` are updated in functions `allocuvm` and `deallocuvm`.

Modifications to other functions :

- `page_fault_handler()` : Earlier, the only reason for pagefault was swapped out page. However, now pagefault could also occur when a process wishes to modify a shared page. This function distinguishes between the 2 scenarios using `swapbit SP` and `writable bit W`. If pagefault occurs on a page whose `SP` is set, it handles it same as handled in lab4. Otherwise, it extracts the physical addr of shared page (call it `pa`) and its permissions (`pageperm`), allocates new page, transfers contents from shared page to new page, updates `pte` of the calling process to store physical addr of new page and `pageperm` with the `W` bit set and finally updates the entries of `rmap` and `proc_ref` corresponding to `pa` and physical addr of new page. However, if the shared page has `refcount = 1`, then it only sets `W` bit and returns.
- The functions `mappages2` and `mappages3` have been defined by slightly modifying `mappages` function. `Mappages2` differs from `mappages` as it also updates entries of the `rmap` and `proc_ref` at physical addr which is being mapped. `Mappages3` is called when creating mapping for swapped pages of the parent process, it does not update the `rmap` and `proc_ref` structures and does not set the `Present` bit since the pages are swapped.
- `copyuvm()` : This function creates a copy of a parent process's page table for a child process during a fork operation. It handles cases where some of the parent's pages are swapped out and some are not. If the page is in memory, it increments the child process's `RSS`, unsets the `W` bit, and maps the page in the child's page table to the same physical page as that of the parent using `mappages2`. If the page is swapped out (`SP` flag set), it unsets the `W` bit, creates a mapping in the child's page table using `mappages3`. Additionally, the `processReference` attribute of the `swapslot` storing the physical page is updated with a pointer to the child's page table entry.
- `kfree()` : Initially, `kfree` always added the given page to the free list. However, with multiple processes potentially referring to the same physical page, `kfree` was modified to free a page only when the calling process was the only one referring to it (`refCount` falls to 0). This ensures that shared pages are not prematurely freed while they are still being referenced by other processes.

In this way, we augmented the `xv6` code to implement demand paging and copy-on-write(COW) policies.